

CHAPITRE 10: PROGRAMMATION EN JAVASCRIPT

LEÇON 1 : INITIATION AU LANGAGE JAVASCRIPT

Compétences visées : Au terme de cette leçon, l'apprenant sera capable de :

- Énoncer et expliquer les limites du HTML liées à l'interactivité ;
- Énoncer et décrire les avantages et les limites de Javascript ;
- Utiliser la balise `<script>...</script>` ;
- Déclarer les variables en Javascript ;
- Effectuer les conversions de type.

Situations problème : En classe de seconde, nous avons dit que la seule interaction dans une page web était le click sur les liens hypertextes. Cependant, comment faire pour récupérer des valeurs saisies par l'utilisateur ? Ou bien, comment faire pour afficher des messages d'alerte, soumettre les données d'un formulaire, réaliser les opérations et bien d'autres. Ces tâches ne sont pas possibles en programmation HTML. Quel est le langage de programmation qui peut supporter ces tâches ?

Introduction

JavaScript est un langage de programmation de scripts orienté-objet qui s'exécutent côté client. Cela signifie que le programme javascript est interprété et exécuté grâce au navigateur web de la machine du programmeur (machine cliente). Le javascript est un langage de scripts qui peut être inclus dans des pages HTML pour les rendre dynamique. Le logiciel pour interpréter et afficher le résultat d'un programme écrit en est appelé le navigateur web. Grâce à ce langage, il est possible d'écrire des pages web interactives.

1. HISTORIQUE, DEFINITIONS ET LIMITES DU HTML LIEES A L'INTERACTIVITE

1. Historique

Javascript a été mis au point par Netscape en 1995 pour enrichir ou apporter des améliorations au html et donner plus d'interactivité tels que la création des boutons, l'affichage des boîtes de dialogue, la récupération des informations du formulaire, la réalisation des opérations, la gestion des événements tels que le clic ou le déplacement de la souris, etc.

2. Définitions

Javascript est un langage de scripts orienté-objet qui peut être incorporé dans un document html pour apporter plus d'interactivité aux pages web.

Un **script** est un ensemble d'instructions intégrer dans le code d'une page html et interpréter par le navigateur web du client.

Un navigateur web est un logiciel d'application permettant d'interpréter et d'afficher le résultat d'un programme javascript.

3. Limites du html liées à l'interactivité

L'interactivité correspond au changement observable suite à une action (le click ou le déplacement de la souris, la saisie de données dans une zone de texte, etc.) effectuée par l'utilisateur dans la page web. Or le seul

changement observable dans une page web apparaît lorsque l'utilisateur click sur le lien hypertexte. Ce qui ne satisfait pas toujours l'utilisateur. Ainsi, les limites du html liées à l'interactivité peuvent être :

- Le html ne permet pas d'afficher les boîtes de dialogue ;
- Le html ne permet pas de soumettre les informations saisies dans le formulaire ;
- Le html ne permet pas le contrôle des données saisies dans le formulaire ;
- Le html ne permet pas d'effectuer les calculs ;
- Le html ne prend pas en compte la gestion des événements tels que le click ou le déplacement de la souris ;

II. AVANTAGES ET LIMITES DU JAVASCRIPT

1. Avantages du javascript

L'avantage principal du javascript est qu'il permet de rendre les pages web interactives ou dynamiques. En effet, le javascript est important car il permet :

- La vérification des champs du formulaire ;
- La validation du formulaire ;
- L'affichage des messages d'alerte ;
- La gestion des événements tels que le click ou le déplacement de la souris ;
- La récupération des valeurs entrées par l'utilisateur pour exécuter des commandes ;
- Réaliser des opérations ;
- Les fonctions javascripts s'exécutent directement grâce au navigateur présent sur votre machine. Etc.

Par ailleurs, le javascript contient aussi des limites.

2. Limites du javascript

Comme limites, nous avons :

- Javascript n'est pas compatible à tous les navigateurs web ;
- Le code javascript n'est sécurisé. Il est visible par tout le monde ;
- Javascript ne permet d'écrire ou de lire sur le disque dur ;
- Les erreurs en javascript sont difficiles à détecter à cause de l'absence d'un débogueur ;
- Javascript ne permet pas de faire des bases de données ;
- Javascript ne permet pas de réaliser des forums de discussion.

3. Différences entre le langage javascript et le langage java

Le tableau ci-après présente quelques différences entre les langages javascript et java :

JavaScript	Java
Langage interprété	Langage pseudo-compilé
S'intègre toujours au code html	Ne s'intègre pas toujours dans le code html
Accessibilité du code	Confidentialité du code
Langage peu évolué et peu typé	Langage très évolué et fortement typé

III. INSERTION DU CODE JAVASCRIPT DANS LA PAGE HTML

Tout script javascript est délimité par les balises `<script>` et fin `</script>`. La syntaxe générale du code javascript est définie par l'une des deux méthodes suivantes :

Méthode 1 :

```
<script language="javascript">  
    Ici le code javascript ;  
</script>
```

Méthode 2 :

```
<script type="text/javascript">  
    Ici le code javascript ;  
</script>
```

Le code javascript peut être inséré à trois endroits par exemple :

- Dans l'en-tête du document html entre les balises <head>...</head>. **Exemple** :

```
<html>
  <head>
    <title>Titre de la page</title>
    <script language="javascript">
      Placer le code javascript ici ;
    </script>
  </head>
  <body>
  </body>
</html>
```

- Dans le corps du document html entre les balises <body>...</body>. **Exemple** :

```
<html>
  <head>
    <title>Titre de la page</title>
  </head>
  <body>
    <script type="text/javascript">
      Placer le code javascript ici ;
    </script>
  </body>
</html>
```

- Dans un fichier externe portant l'extension .js. Dans ce cas, il faut écrire dans l'en-tête du code html le lien vers le fichier. Par exemple : `<script type="text/javascript" src="MonFichier.js"></script>`. Le principal avantage d'écrire le code javascript dans un fichier externe est que ce code peut être réutiliser pour d'autres pages web. C'est la méthode la plus recommandée.

NB : Pour insérer les commentaires sur une seule ligne dans le code javascript, il faut utiliser le double slash //. Lorsque le commentaire dépasse une ligne, il faut utiliser /* et */.

Exemple de code javascript :

/* Le code javascript suivant permet d'afficher le message « Bonne et heureuse année académique 2019-2020 à tous les élèves. » en utilisant la méthode **document.write()**. */.

```
<html>
  <head>
    <title> Voici une page contenant du Javascript</title>
  </head>
  <body>
    <script language="Javascript">
      document.write ("Bonne et heureuse année académique 2019-2020 à tous les élève"); //Permet d'afficher le
message.
    </script>
  </body>
</html>
```

IV. Opérateurs

i. Opérateurs arithmétiques

Comme en mathématique, en JavaScript nous avons les operateurs : addition (+), soustraction (-), multiplication(*), le reste de la division modulo (%).

ii. Opérateurs de comparaison

Les operateurs de comparaisons en JavaScript sont :

traduisant l'égalité	==
traduisant l'inégalité	!=
supérieur ou égal	>=
inferieur ou égal	<=
inferieur	<
supérieur	>
pour traduire l'affectation	=

iii. **Opérateurs logique**

Les opérateurs logiques en JavaScript sont :

traduisant le ET logique	&&
traduisant le OU logique	
traduisant le NON logique	!
traduisant	===

V. **DECLARATION DES VARIABLES EN JAVASCRIPT**

Une **variable** est un objet mémoire dont le contenu peut être modifié au cours de l'exécution du script. En javascript, le nom d'une variable doit respecter les règles suivantes :

- Il doit commencer par une lettre majuscule, minuscule ou par le caractère underscore ;
- Il ne doit pas commencer par un chiffre ;
- Il ne doit pas contenir des espaces ;
- Il ne doit pas être un mot réservé comme : **Boolean, char, int, public, if, for, return, while, float, private, function**, etc.

En outre, javascript est sensible à la casse ! C'est-à-dire que javascript fait la différence entre les majuscules et les minuscules, les caractères accentuées et les caractères non-accentués. **Exemples** : « **M**variable » est différente de « **m**variable » de même que « **é**lève » est différent de « **e**leve ».

En javascript, il existe deux types de déclarations de variables : **la déclaration explicite** et **la déclaration implicite**.

IV.1 **Déclaration explicite et déclaration implicite**

Pour déclarer explicitement une variable, il faut utiliser le mot clé « **var** ». Il permet d'indiquer de façon rigoureuse que c'est une variable. Le tableau suivant présente quelques exemples de déclarations explicites.

Var x ; // déclaration explicite

Var x=18 ; //déclaration explicite avec affectation de valeur.

Pour déclarer une variable de manière implicite, le mot-clé « var » n'est pas utilisé. Le tableau suivant présente quelques exemples de déclarations implicites.

Pour déclarer implicitement une variable, il suffit d'écrire le nom de la variable suivie de caractère = (égal) et de la valeur à affecter. **Exemple** : `chaine="bonjour"`. Dans la déclaration implicite, le soin est laissé au navigateur de déterminer qu'il s'agit d'une déclaration de variable.

NB : Même si une déclaration implicite est tout à fait reconnue par le navigateur, il est plus rigoureux de déclarer les variables de façon explicite avec le mot-clé « **var** ».

En javascript, nous pouvons aussi déclarer les variables en utilisant le constructeur « new ». Ce dernier permet de déclarer les variables de même type que les types de base de javascript. Les types de base en javascript sont : Nombre (entier ou à virgule), Booléen et Chaîne de caractères.

Exemples: `var x = new Number (); var y = new String ("Bonjour");`

IV.2 Portée (visibilité) des variables

Selon l'endroit où une variable est déclarée, celle-ci pourra être accessible (visible) de partout dans le script ou bien uniquement dans une portion confinée du code. On parle de « **portée** » d'une variable. La **portée d'une variable** représente la portion du code où la variable est visible.

Lorsqu'une variable est déclarée sans le mot-clé **var**, c'est-à-dire **de façon implicite**, elle est accessible de partout dans le script (n'importe quelle fonction du script peut faire appel à cette variable). On parle alors de **variable globale**.

La portée d'une variable déclarée **de façon explicite** (précédée du mot-clé *var*), dépend de l'endroit où elle est déclarée :

- Une variable déclarée au début du script, avant toute fonction, sera **globale**. Elle peut être utilisée n'importe où dans le script ;
- Une variable déclarée explicitement dans une fonction aura une portée limitée à cette seule fonction, c'est-à-dire qu'elle est inutilisable ailleurs. On parle alors de **variable locale**.

Voici deux exemples permettant de l'illustrer :

```
<SCRIPT language="Javascript">
  <!--
    var a = 12;
    var b = 4;
    function MultipliePar2(b) {
      var a = b * 2;
      return a;
    }
    document.write("Le double de ",b," est ",MultipliePar2(b));
    document.write("La valeur de a est ",a);
  // -->
</SCRIPT>
```

Dans l'exemple ci-dessus, la variable *a* est déclarée explicitement en début de script, ainsi que dans la fonction. Voici ce qu'affiche ce script :

Le double de 4 est 8
La valeur de a est 12

Voici un autre exemple dans lequel *a* est déclarée implicitement dans la fonction :


```
<SCRIPT language="Javascript">
<!--
    var a = 12;
    var b = 4;
    function MultipliePar2(b) {
        a = b * 2;
        return a;
    }
    document.write("Le double de ",b," est ",MultipliePar2(b));
    document.write("La valeur de a est ",a);
    // -->
</SCRIPT>
```

Voici ce qu'affiche ce script :
Le double de 4 est 8
La valeur de a est 8

VI. TYPES DE DONNÉES DES VARIABLES JAVASCRIPT

Un **type de données** correspond à un ensemble de valeurs qu’une variable peut prendre. En javascript, il n’est pas nécessaire de déclarer le type de variable contrairement à des langages évolués tels que le C ou le java pour lesquels il faut toujours indiquer le type de variables. Javascript utilise quatre types de données :

- **Number** : il s’agit des nombres entiers ou à virgule tels que 22 ou 3.14 ;
- **String** : il s’agit des chaines de caractères tels que "bonjour" ;
- **Boolean** : il s’agit des variables qui peuvent prendre deux valeurs : "**True**" si le résultat est vrai ou "**False**" si le résultat est faux. Elles permettent de vérifier une condition ;
- **Null** : il s’agit des variables qui ne contiennent aucune donnée.

VII. CONVERSION DE TYPES

Par défaut, les variables en javascript sont de types chaine de caractères (String). Il sera souvent nécessaire de convertir ces variables en types entier ou en type réel.
Pour convertir une variable en type entier, il faut utiliser la fonction **parseInt()**. Il faut utiliser la fonction **parseFloat()** pour convertir une chaine en type réel.

VI.1 FONCTION PARSEINT()

La fonction *parseInt ()* permet de convertir une variable passée en paramètre (soit en tant que chaîne de caractère, soit en tant que nombre dans la base précisée en second paramètre) en nombre entier (en base décimale). La syntaxe de la fonction *parseInt()* est la suivante : **parseInt(chaine[, base])** ;

Pour que la fonction *parseInt()* retourne un entier, la chaine passée en paramètre doit commencer par des caractères valides : c'est-à-dire les chiffres **[0-9]** ou le préfixe hexadécimal **0x**, et/ou les caractères **+**, **-**, **E** et **e**. Dans le cas contraire la fonction *parseInt()* retournera la valeur **NaN (Not a Number)**.

Si les caractères suivants ne sont pas valides, ils seront ignorés par la fonction *parseInt()*. Si la chaîne passée en paramètre représente un nombre possédant une partie littérale, celle-ci sera tronquée.

Le paramètre "**base**" est un entier facultatif permettant de préciser la **base** devant être utilisée pour interpréter la chaîne. Il vaut **10** par défaut. Si le paramètre **base** n'est pas précisé (ou s'il est fixé à la valeur 10), la base utilisée sera la base décimale ; la base sera **16** si la chaîne commence par **0x**, elle sera **8** si la chaîne commence par **0**. Le tableau suivant donne des exemples d'utilisation de la fonction *parseInt()*

Exemples	Résultats
parseInt("128.34 ")	128
parseInt("Bonjour ")	NaN
parseInt("0284,8 ")	284
parseInt("12.3E-6")	12

parseInt("AF8BEF")	NaN
--------------------	-----

VI.2 La fonction parseFloat()

La fonction *parseFloat()* permet de convertir une variable passée en paramètre en nombre en virgule flottante (**nombre avec une partie décimale**). Sa syntaxe est la suivante : **parseFloat(chaine)** ;

Pour que la fonction *parseFloat()* retourne un flottant, la chaîne passée en paramètre doit commencer par des caractères valides. C'est-à-dire les chiffres [0-9] et/ou les caractères +, -, *E* et *e*. Dans le cas contraire la fonction *parseFloat()* retournera la valeur *NaN* (*Not a Number*).

Si les caractères suivants ne sont pas valides, ils seront ignorés par la fonction *parseFloat()*. Si la chaîne passée en paramètre représente un nombre possédant une partie littérale, celle-ci sera tronquée. Le tableau suivant donne des exemples d'utilisation de la fonction *parseFloat()* :

Exemples	Résultats
parseFloat("128.34") ;	128.34
parseFloat("128,34") ;	128
parseFloat("12.3E-6") ;	0.0000123
parseFloat("Bonjour") ;	NaN
parseFloat("24.568Bonjour38") ;	24.568
parseFloat("Bonjour28.34") ;	NaN
parseFloat("AB8BEF") ;	NaN
parseFloat("0234") ;	234

Conclusion

Javascript est un langage de programmation de script qui s'exécute grâce au navigateur web. Il permet d'apporter des améliorations au html tels que l'affichage des boites de dialogue. Toutefois, le javascript est un langage de programmation basique dont la prise en main est nécessaire. \$